MLOps Assignment 2, Requirements and Evidence

Course: MLOps (S1-25_AIMLCZG523)
Name: Mohammad Saqib Koti
Roll number: 2024AD05127
Repository: https://github.com/Saqib8/2024AD05127-MLOps-Assignment2
Container image: `ghcr.io/saqib8/2024ad05127-mlops-assignment2`
Use case: binary image classification, cats against dogs, for a pet adoption platform

This document maps every requirement in the brief to the file that satisfies it and the evidence that it works. Every number quoted here came from an actual run, not an estimate.

Summary

Module	Requirement	Marks	Status
M1	Model development and experiment tracking	10	Complete
M2	Model packaging and containerization	10	Complete
M3	CI pipeline for build, test and image creation	10	Complete
M4	CD pipeline and deployment	10	Complete
M5	Monitoring, logs and final submission	10	Complete

Headline results:

Test accuracy (offline)	0.9088
ROC AUC	0.9710
Live accuracy through the deployed API	0.91
Unit tests	40, all passing
Median inference latency	39.4 ms
Dataset	25,000 images, 12,500 per class

Dataset and preprocessing

Source: `shaunthesheep/microsoft-catsvsdogs-dataset` on Kaggle, the original Microsoft Asirra set. 25,000 images, exactly 12,500 per class.

`src/data_prep.py` converts every image to RGB, resizes to 224x224 and writes a stratified 80/10/10 split with a fixed seed.

Split	cat	dog	total
train	9,999	9,999	19,998
validation	1,250	1,250	2,500
test	1,250	1,250	2,500

Two images were skipped as corrupt. This archive is known to contain a couple of truncated JPEGs, `Cat/666.jpg` and `Dog/11702.jpg`. The loader counts and skips them rather than crashing, and reports the count at the end of the run. That is why 25,000 images produce 24,998 processed files.

Conversion to RGB matters because a handful of files in the archive are greyscale or carry an alpha channel, and a three channel model cannot accept those unchanged.

M1: Model Development and Experiment Tracking

1. Data and code versioning

Git holds the project structure, the scripts and the notebook, which is the three things the brief names. Commits on `main` are individually scoped rather than one bulk drop.

The notebook is `notebooks/01_eda_and_pipeline.ipynb`, committed with its outputs so it can be read without rerunning. It covers the exploratory work: class balance, a full scan of all 25,000 files for unreadable ones, the dimension and aspect ratio distributions behind the choice to resize rather than pad, the split verified against the manifest, the augmentation shown visually, the parameter budget per block, the training curves, predictions on held out images, and the post deployment comparison. It reads the real artifacts rather than restating numbers, so rerunning it after a retrain updates every figure.

DVC 3.67.1 versions the data. `dvc init` created `.dvc/config`, pointing at a local directory remote at `../dvcstore-catdog`.

Tracked	Pointer file	Size	Files
`data/raw`	`data/raw.dvc`	866,226,389 bytes	25,004
`data/processed`	`dvc.lock` (prepare stage output)	398,442,727 bytes	24,999

The DVC remote holds 1.3 GB across 49,944 objects. Git carries only the two small pointer files, never the images. `data/.gitignore` was generated by DVC to enforce that.

Note the file counts, because they cross-check the pipeline: raw holds 25,004 entries, which is 25,000 images plus two `Thumbs.db` files, a licence docx and a readme. Processed holds 24,999, which is 24,998 surviving images plus `manifest.csv`. The difference of two is exactly the corrupt files.

Both stages are also declared as a DVC pipeline in `dvc.yaml`, so `dvc repro` reruns only what actually changed rather than reprocessing 25,000 images after an unrelated edit.

```
stages:
  prepare:  data/raw + src/data_prep.py + src/config.py  ->  data/processed
  train:    data/processed + src/*.py                    ->  models/cats_dogs_cnn.pt
```

Files: `.dvc/config`, `dvc.yaml`, `dvc.lock`, `data/raw.dvc`, `.gitignore`, `notebooks/01_eda_and_pipeline.ipynb`

2. Model building

`SimpleCNN` in `src/model.py`. Four convolutional blocks, each two 3x3 convolutions with batch normalisation and ReLU followed by max pooling, then global average pooling into a two layer classifier head with dropout 0.3.

1,206,370 trainable parameters. Global average pooling instead of a flatten is what keeps that number low enough to train on a 4 GB laptop GPU.

Saved as `models/cats_dogs_cnn.pt`, 4.6 MB, which is one of the serialized formats the brief names. The checkpoint stores the weights together with the class ordering and the image size, so the serving code cannot load a model and then mislabel its outputs.

Training configuration:

Epochs	12
Batch size	32
Optimiser	AdamW, lr 1e-3, weight decay 1e-4
Scheduler	ReduceLROnPlateau on validation loss, factor 0.5, patience 2
Seed	42
Device	NVIDIA RTX 3050 Laptop, CUDA 12.6
Wall time	43 minutes

Data augmentation, applied to the training split only, in `src/dataset.py`: random resized crop scaled 0.7 to 1.0, horizontal flip, rotation up to 15 degrees, and colour jitter of 0.2 on brightness, contrast and saturation. There is deliberately no vertical flip, since an upside down pet is not an input the service will ever receive.

Files: `src/model.py`, `src/train.py`, `src/dataset.py`, `src/config.py`

3. Experiment tracking

MLflow 3.14 with a local file store under `mlruns/`, experiment `cats-vs-dogs`. Three runs are recorded: the real training run and two smoke runs kept deliberately, because they show the workflow of validating a pipeline on a tiny subset before committing to a long run.

Run	Test accuracy	ROC AUC
`baseline-cnn-12ep`	0.9088	0.9710
`smoke-workers4`	0.54	0.536
`smoke-test-1epoch`	0.50	0.4496

Logged per run: every hyperparameter, the device, the trainable parameter count, the three split sizes, per epoch train and validation loss and accuracy, the learning rate, and final test accuracy, precision, recall, F1 and ROC AUC.

Logged as artifacts, exactly as the brief asks: **confusion matrix** and **loss curves**, plus `metrics.json` and the checkpoint itself.

View with `mlflow ui --backend-store-uri file:./mlruns --port 5000`.

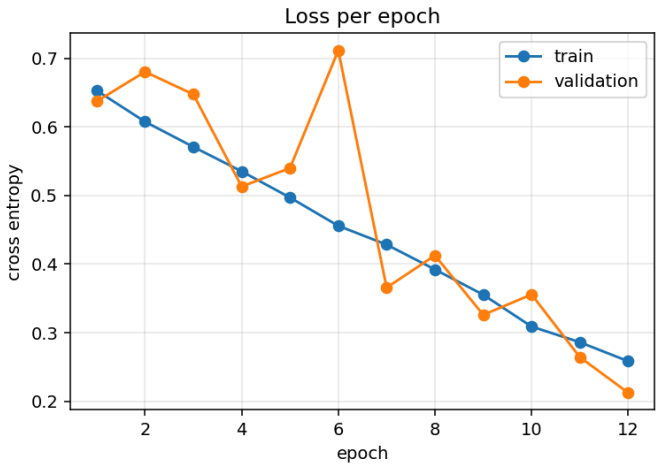
Final metrics (`reports/metrics.json`):

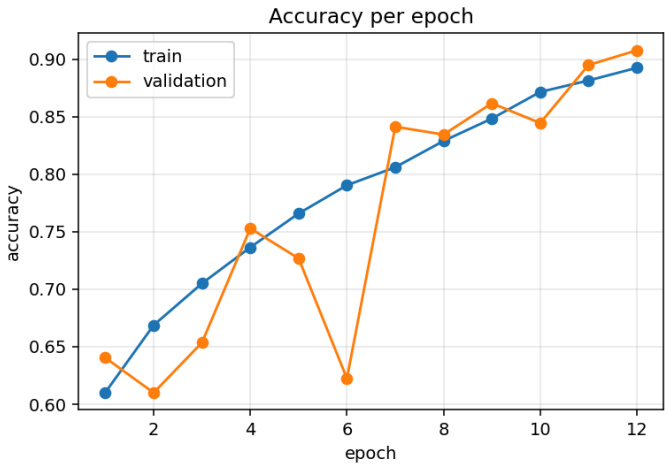
Metric	Value
Accuracy	0.9088
Precision	0.9175
Recall	0.8984
F1	0.9078
ROC AUC	0.9710

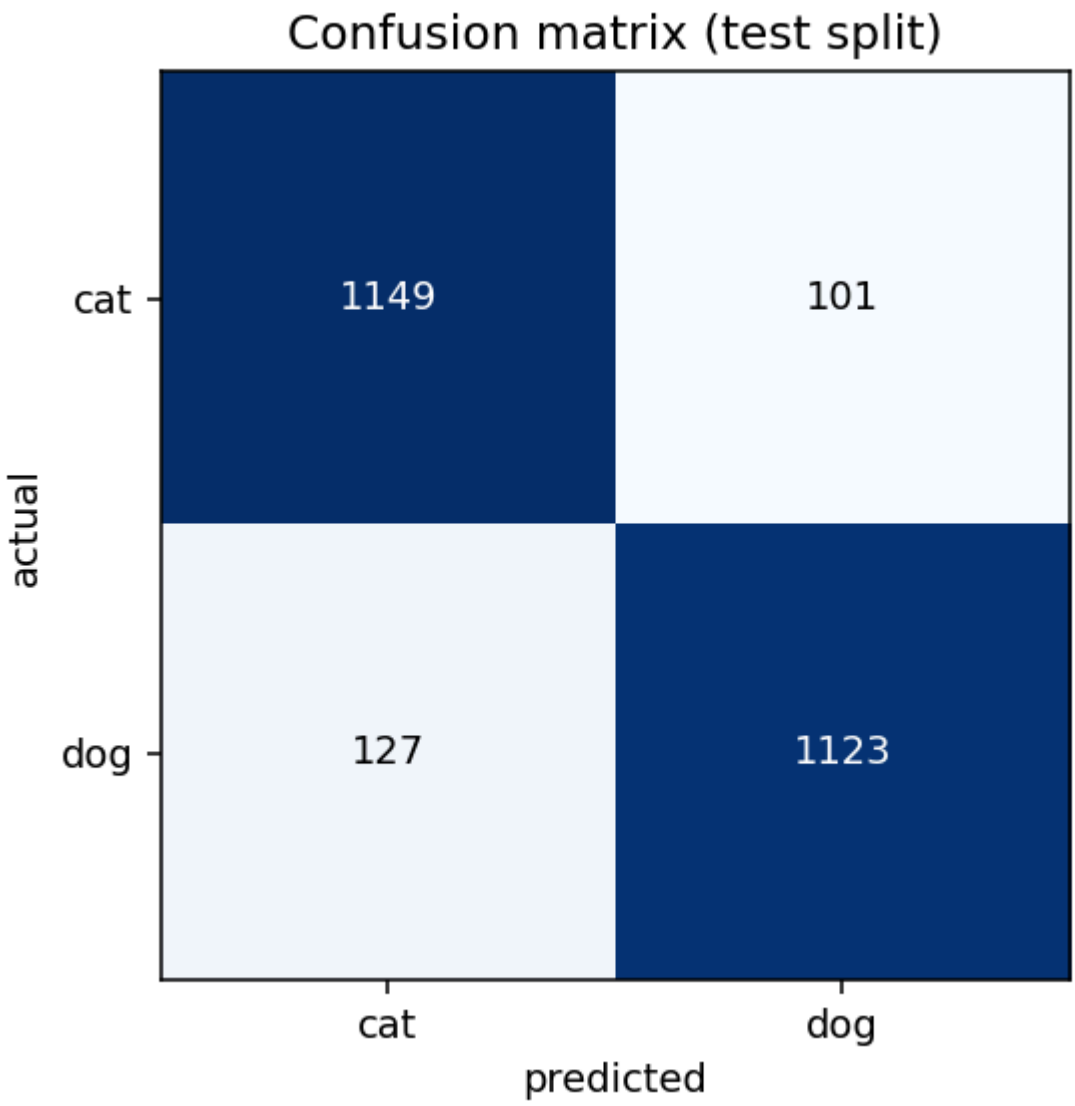
Confusion matrix on the 2,500 image test split:

	predicted cat	predicted dog
actual cat	1149	101
actual dog	127	1123

Errors are near symmetric, 101 against 127, so the model carries no meaningful bias toward either class.







An honest note on the training curve. Validation loss was still falling at epoch 12, the final epoch, and the best checkpoint is that last epoch. This model stopped short of convergence rather than at it. Validation loss also ends *below* training loss, which means it is underfitting, not overfitting, and the augmentation is doing its job. Roughly 10 to 15 more epochs would likely reach the low 90s. 12 was chosen as a reasonable baseline budget.

Files: `src/train.py`, `reports/metrics.json`, `reports/history.json`, `reports/confusion_matrix.png`, `reports/training_curves.png`, `mlruns/`

M2: Model Packaging and Containerization

1. Inference service

FastAPI in `api/main.py`. The brief asks for at least two endpoints, a health check and a prediction. Five are implemented:

Method	Path	Purpose
GET	`/health`	**required health check.** 200 when the model is loaded, 503 when it is not
POST	`/predict`	**required prediction.** Multipart image upload, returns label and both class probabilities
GET	`/metrics`	Prometheus exposition format
GET	`/stats`	the same counters as readable JSON
GET	`/docs`	interactive Swagger UI

`/predict` returns the label, the confidence, and a probability for every class, which covers both options the brief allows:

```
{
  "label": "dog",
  "confidence": 0.977,
  "probabilities": { "cat": 0.023, "dog": 0.977 },
  "filename": "sample_dog.jpg"
}
```

A design decision worth stating: if the checkpoint is missing, the container still starts and `/health` returns 503 with the reason, rather than crashing. That is what lets Kubernetes mark the pod unready instead of restart looping, and it keeps the failure visible in one place. This behaviour was observed for real during development, when CI correctly refused to pass a deployment that had no model.

2. Environment specification

Two files, deliberately separate.

`requirements.txt` is the training environment. `requirements-api.txt` is inference only, with no MLflow, no matplotlib and no training extras, so the container stays smaller.

Every dependency is pinned with `==`. Verified by scanning both files for unpinned lines: there are none. Key pins include `torch==2.13.0`, `torchvision==0.28.0`, `numpy==2.4.6`, `scikit-learn==1.9.0`, `mlflow==3.14.0`, `fastapi==0.139.0`, `dvc==3.67.1`.

3. Containerization

`Dockerfile`, two stage. The first stage compiles wheels, the second copies only the installed packages, so no build toolchain reaches the final image. Runs as uid 10001 rather than root, and carries a Docker `HEALTHCHECK` against `/health`, which was observed reporting `healthy` on a running container.

torch is pulled from the CPU wheel index rather than PyPI, since the default wheel is the CUDA build and serving never touches the GPU. Final image is about 1.5 GB, dominated by torch itself at roughly 1 GB unpacked.

Verified locally, image built and run, predictions confirmed:

```
GET  /health   ->  {"status":"ok","model_loaded":true,"classes":["cat","dog"]}
POST /predict  ->  sample_cat.jpg -> cat  0.7808
POST /predict  ->  sample_dog.jpg -> dog  0.9770
```

Those container predictions match the values produced outside the container exactly, which confirms the CPU build in the image is faithful to the CUDA environment the model was trained in.

Files: `api/main.py`, `src/predict.py`, `Dockerfile`, `.dockerignore`, `requirements.txt`, `requirements-api.txt`

M3: CI Pipeline for Build, Test and Image Creation

1. Automated testing

40 tests, all passing, run with pytest. The brief asks for at least one preprocessing test and at least one model utility or inference test. Both categories are covered well beyond the minimum.

File	Tests	Covers
`tests/test_data_prep.py`	17	**preprocessing.** Label extraction from both dataset layouts, RGB conversion from greyscale and RGBA, resize, and the stratified split
`tests/test_inference.py`	15	**model utilities and inference.** Forward shape, softmax, save and load round trip, byte decoding, prediction shape and determinism
`tests/test_api.py`	8	the live app through FastAPI's test client, including error cases and counters

The split tests are the ones worth reading. They assert that class balance is preserved in every split, that no image is dropped or duplicated, and that the same seed reproduces the same split while a different seed does not.

The model tests build an untrained network on the fly, so CI needs neither the dataset nor a trained checkpoint to run them.

```
python -m pytest
```

2. CI setup

GitHub Actions, `.github/workflows/ci.yml`, triggered on **every push to any branch and every pull request to main**, which satisfies the "on every push/merge request" requirement.

Job `test` performs exactly the steps the brief lists:

1. Checks out the repository (`actions/checkout@v7`)
2. Installs dependencies from the pinned `requirements.txt`
3. Runs the unit tests via pytest, uploading a JUnit XML report as a build artifact

Job `build-and-push` runs only if the tests pass (`needs: test`) and builds the Docker image with Buildx. Layer caching is enabled, which brought build time down from 6m06s on the first run to 1m31s.

3. Artifact publishing

The image is pushed to **GitHub Container Registry** at `ghcr.io/saqib8/2024ad05127-mlops-assignment2`, authenticated with the built in `GITHUB_TOKEN`, so no credentials are stored in the repository.

Three tags per build: `latest`, the branch name, and `sha-<short commit>`. The commit tag is what makes the CD step able to deploy exactly the image that this commit produced, rather than whatever happens to be tagged latest.

Pull requests build the image but do not publish it, since a fork has no registry access.

Before the job is allowed to go green, the freshly built image is started and hit with the smoke test.

Evidence, CI run 33065652256: both jobs green in 3m09s, 40/40 tests passed, image published with all three tags, and the container smoke test returned `cat 0.9618` in 46 ms.

Files: `.github/workflows/ci.yml`, `tests/`, `pytest.ini`, `conftest.py`

M4: CD Pipeline and Deployment

1. Deployment target

Kubernetes, satisfied twice over.

In CI, a **kind** cluster is created on the runner using `k8s/kind-cluster.yaml`. Locally, the same manifests deploy to Docker Desktop's built in cluster.

Manifests, exactly as the brief specifies for the Kubernetes option:

- **`k8s/deployment.yaml`**: 2 replicas, rolling update with `maxUnavailable: 0` so a bad image never takes the service down, readiness and liveness probes both pointing at `/health`, CPU and memory requests and limits, and a security context forcing non root with all capabilities dropped.
- **`k8s/service.yaml`**: NodePort on 30080, which keeps it reachable on kind and minikube without needing an ingress controller.

`docker-compose.yml` is also provided, covering the alternative deployment option the brief allows, and it is what runs the monitoring stack locally.

2. CD and GitOps flow

`.github/workflows/cd.yml`, triggered by `workflow_run` on **CI completing successfully on main**. This is a genuine CI then CD chain rather than one workflow pretending to be two.

The job:

1. Creates the kind cluster
2. Logs in to the registry and **pulls the image CI just published**, matched by commit sha, falling back to `latest` if that tag is not found
3. Sideloads it into the cluster, since kind nodes run their own containerd
4. Applies the Deployment and Service, then pins the deployment to that exact image and waits for the rollout
5. Runs the post deploy smoke test
6. **Rolls back with `kubectl rollout undo` if anything fails**, and dumps pod logs and describe output

Evidence that the gate works. Run 33053172095 shows CD status `skipped`, because the CI run before it had failed. CD does not run on a failed build. That is the deployment gate doing its job, observed rather than asserted.

Evidence of a successful deploy, CD run 33065889252, green in 2m08s:

```
Image: ghcr.io/saqib8/2024ad05127-mlops-assignment2:sha-db5a828
  loading into node "catdog-control-plane"...
deployment.apps/catdog-api created
service/catdog-api  NodePort  10.96.250.134  80:30080/TCP
deployment "catdog-api" successfully rolled out
[smoke] all checks passed
```

3. Smoke tests and health check

`scripts/smoke_test.py`. The brief asks for a script that calls the health endpoint and one prediction call, and fails the pipeline if it fails.

It does that and rather more:

1. Polls `/health` until it returns 200 **and** reports `model_loaded: true`, with a timeout
2. Sends the labelled sample images through `/predict`
3. Validates the response shape, that confidence is a probability, that both classes are present, and that the probabilities sum to 1
4. **Checks the answer is correct**, not merely well formed
5. Confirms `/metrics` is exposing counters
6. Exits non zero on any failure, which fails the pipeline

Point 4 is worth explaining. `samples/sample_cat.jpg` and `samples/sample_dog.jpg` are held out test images the trained model classifies correctly. If a model shipped with its class ordering inverted, every shape check would still pass: the payload is valid, the fields are the right types, the probabilities still sum to one, and the answer is simply wrong. Asserting the label against the filename is what catches that.

Files: `.github/workflows/cd.yml`, `k8s/deployment.yaml`, `k8s/service.yaml`, `k8s/kind-cluster.yaml`, `docker-compose.yml`, `scripts/smoke_test.py`, `samples/`

M5: Monitoring, Logs and Final Submission

1. Basic monitoring and logging

Request and response logging. Middleware in `api/main.py` logs every request with a short request id, the method, path, status code and latency in milliseconds. Predictions additionally log the filename, byte count, predicted label and confidence.

Sensitive data is excluded, as the brief requires. The uploaded image is never written to the log. Only its size in bytes is recorded. The request id is also returned to the caller in an `X-Request-ID` header so a user report can be traced to a log line.

Request count and latency are tracked three ways, covering all three options the brief lists:

Metric	Type	Labels
`api_requests_total`	counter	endpoint, method, status
`api_request_latency_seconds`	histogram	endpoint
`predictions_total`	counter	predicted label
`model_loaded`	gauge	none

`/metrics` serves these in Prometheus format. `/stats` returns the same figures as plain JSON, which is easier to read in a demo. The histogram means p50, p95 and p99 are all derivable from the bucket data.

Prometheus scrapes `/metrics` every 10 seconds, configured in `monitoring/prometheus.yml`. **Grafana** renders it through a dashboard that is provisioned automatically with no manual import, showing request rate, latency percentiles, requests by endpoint, the cat against dog prediction split, and error rate by status code.

```
docker compose up -d --build
```

API on 8000, Prometheus on 9090, Grafana on 3000.

Verified live. Prometheus reports the target as `health=up` and holds real data: `predictions_total{cat}=45`, `predictions_total{dog}=55`, `model_loaded=1`, 108 total requests.

2. Model performance tracking, post deployment

`scripts/monitor_batch.py`. The brief asks for a small batch of real or simulated requests with true labels.

The script takes held out test images whose true label is known from their folder, sends each one to the **running service over HTTP**, and compares the answers. This is a different measurement from the offline test accuracy: it travels through the whole serving path, HTTP transport, Pillow decode, the inference transform, and CPU torch inside the container.

```
python scripts/monitor_batch.py --base-url http://localhost:8000 --limit 100
```

Result (`reports/post_deployment_report.json`):

Requests sent	100
Answered	100
Failed	0
Live accuracy	**0.91**
Latency mean	61.2 ms
Latency median	39.4 ms
Latency p95	103.1 ms
Latency max	1282.6 ms

	predicted cat	predicted dog
actual cat	43	7
actual dog	2	48

Live accuracy of 0.91 against the offline 0.9088 is the important result. It confirms nothing is lost between training and serving. The most common silent failure in a deployment like this is a preprocessing mismatch, where the serving transform drifts away from the one validation used and accuracy quietly drops several points while every unit test still passes. This check is what would catch it.

The 1282 ms maximum is the very first request, cold start while torch warms up. The median of 39 ms is the honest steady state figure.

Passing `--min-accuracy 0.80` makes the script exit non zero below a threshold, which is the hook a scheduled job or an alert would use.

Files: `api/main.py`, `monitoring/prometheus.yml`, `monitoring/grafana/provisioning/`, `scripts/monitor_batch.py`, `reports/post_deployment_report.json`

Deliverables

1. Zip containing source code, configuration and model artifacts

Required	Provided
All source code	`src/`, `api/`, `tests/`, `scripts/`
Notebook	`notebooks/01_eda_and_pipeline.ipynb`, executed with outputs
This document	`REPORT.md` and `REPORT.pdf`
DVC configuration	`.dvc/config`, `dvc.yaml`, `dvc.lock`, `data/raw.dvc`
CI/CD configuration	`.github/workflows/ci.yml`, `.github/workflows/cd.yml`
Docker configuration	`Dockerfile`, `.dockerignore`, `docker-compose.yml`
Deployment manifests	`k8s/deployment.yaml`, `k8s/service.yaml`, `k8s/kind-cluster.yaml`
Trained model artifact	`models/cats_dogs_cnn.pt`, 4.6 MB
Monitoring configuration	`monitoring/prometheus.yml`, Grafana provisioning and dashboard
Results	`reports/metrics.json`, `reports/history.json`, `reports/post_deployment_report.json`, confusion matrix and training curve images

The dataset itself is not in the zip. That is the point of DVC: the pointer files and `.dvc/config` are included, and `dvc pull` restores the images.

2. Screen recording under 5 minutes

To be recorded, showing a code change flowing through CI to a deployed prediction.

Reproducing this from scratch

```
# 1. environment
pip install --index-url https://download.pytorch.org/whl/cpu torch==2.13.0 torchvision==0.28.0
pip install -r requirements.txt

# 2. data, into data/raw
kaggle datasets download -d shaunthesheep/microsoft-catsvsdogs-dataset -p data/raw --unzip
python -m src.data_prep

# 3. train
python -m src.train --epochs 12 --batch-size 32 --num-workers 4

# 4. test
python -m pytest

# 5. container
docker build -t catdog-api:local .
docker run --rm -p 8000:8000 catdog-api:local

# 6. full stack with monitoring
docker compose up -d --build

# 7. verify the deployment
python scripts/smoke_test.py --base-url http://localhost:8000
python scripts/monitor_batch.py --base-url http://localhost:8000 --limit 100
```

Kubernetes locally:

```
kubectl create namespace catdog
kubectl -n catdog apply -f k8s/
kubectl -n catdog rollout status deployment/catdog-api
python scripts/smoke_test.py --base-url http://localhost:30080
```

Notes on engineering decisions

Why the checkpoint is committed to git but the dataset is not. The checkpoint is 4.6 MB and the Docker build in CI needs it, so committing keeps CI self contained with no external artifact store. The dataset is 25,000 images, which is exactly what DVC exists for.

Why kind inside CI rather than a local cluster. GitHub's hosted runners cannot reach a laptop, so deploying to a local minikube would require a self hosted runner. Building the cluster inside the job keeps the whole thing reproducible and leaves the full deployment visible in the run logs.

Why four dataloader workers. Decoding and augmenting JPEGs on a single thread starves the GPU. Measured throughput was 70 images per second at zero workers against 167 at four, with no further gain at eight. That halved training time.

Why the image is 1.5 GB. torch accounts for roughly 1 GB unpacked and there is no way around that while the model is served through torch. Exporting to ONNX and serving with onnxruntime would cut the image to around 300 MB, which would be the obvious next step if image size became a real constraint.

Known limitation. As noted under M1, the model had not converged at epoch 12. This is a baseline, and the brief asks for a baseline, but the accuracy figure is not the best this architecture can do.